

HINT 365 · How it works

The whole service in plain words: what it is for, who does what, where everything runs, how changes, releases, security patches and compliance are handled, and what is free. Updated 02.10.2026, app version 0.1.121.

This is the general document. Every other document is linked in section 15; how to pick the work up again is in section 16. Times are given in Zurich time and in UTC (Zurich is UTC+2 in summer, UTC+1 in winter).

1. What HINT 365 is for

HINT 365 is a **personal diary** on an Android phone. A person records their blood pressure (the top number **SYS**, the bottom number **DIA** and the pulse **PUL**) and keeps the results of their lab reports. The app shows them in tables, charts and a PDF report that can be sent to a doctor.

It is not a medical device. HINT 365 does not measure anything, does not say whether a value is good or bad, does not diagnose and does not give advice. It only keeps track of what the person records. The doctor decides what the values mean. For this reason no colour on any screen means "good" or "bad", and there is no red.

Responsibility. The person checks every value before saving it and against the original report. The owner provides the service as described in the [terms of use](#) (accepted in the app at the first start and at every update) and handles personal data as described in the [privacy policy](#). Those two documents are binding; this one only explains.

2. Who does what

Who	What they do
The person (user)	Installs the app, signs in with Google, accepts the terms, records readings by voice or with a photo, imports lab reports, looks at the charts, sends the report to the doctor, deletes data or the whole account whenever they want.
The owner (Human)	Decides what the app does, the prices and the rules; tests every new version; says "go" before a release; approves any change to the terms of use; owns every account and key (GitHub, Cloudflare, Google, Anthropic).
The developer (Claude)	Writes the code and the documents, runs the tests, prepares each release, checks the error log and the security scans every morning, fixes what it can and proposes the rest to the owner.
GitHub	Keeps the code and the documents, runs the tests and the build, and stores every released app.
Cloudflare	Runs the server and keeps the database, in the European Union.
Google	Lets people sign in with their Google account; later, sells the subscription through Google Play.
Anthropic	Reads the photo of the blood-pressure monitor for the Scan (artificial intelligence), on the owner's account.

3. What the person does, step by step

First time

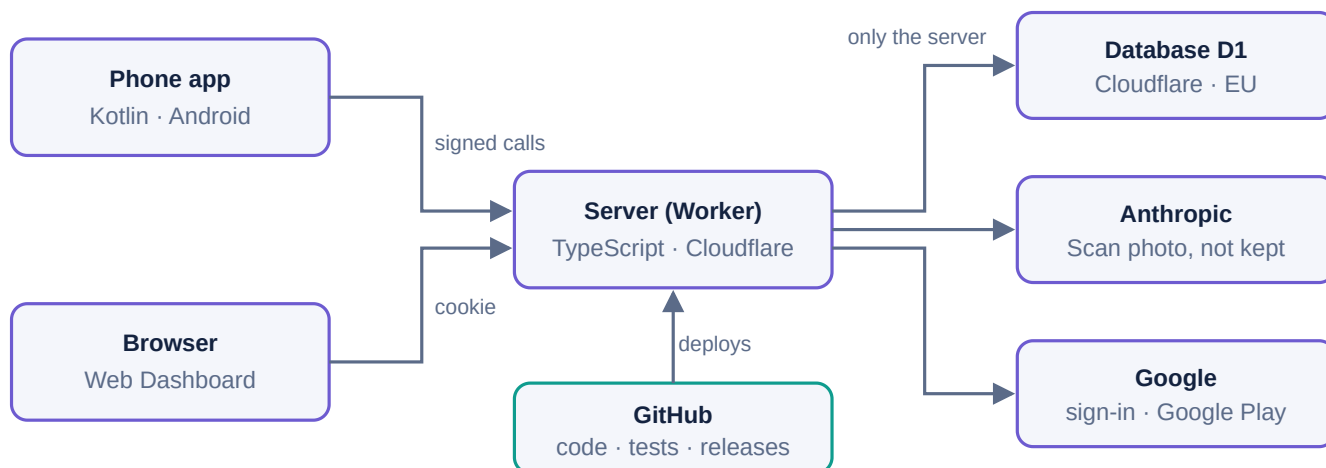
1. **Installs the app** from the download link (later from Google Play).
2. **Signs in with Google.** The Google account is used only to recognise the person on a new phone. The email address is never stored on the server: only a coded fingerprint of the account that cannot be turned back into the email.
3. **Accepts the terms of use** and the storage of health data. Without that the app does not open. The acceptance is recorded as proof.
4. The phone creates its own **private key**, which never leaves the phone. Every request the app sends is signed with it, so the server knows it really comes from that phone.

Every day

- **Opening the app:** the app uses the phone's own lock (fingerprint, face or PIN). HINT 365 has no PIN of its own. If the phone has no lock, the app suggests one, at most every 14 days; it is only a suggestion.
- **Voice** (blue button): the person says the three numbers; the phone turns speech into text; the app finds SYS, DIA and PUL; the person checks them and confirms; only then are they saved.
- **Scan** (violet button with a small magic wand, an AI feature): the person photographs the monitor's display; the photo goes to the server and from there to Anthropic, which reads the three numbers; the photo is not kept anywhere; the person checks the numbers and confirms. Free for 15 days (3 a day), then with the 4.99 US\$ yearly subscription.
- **Lab reports:** the person picks a PDF or a photo of a report; the phone reads it by itself; only the date and the values as printed (no name, no personal details) go to the server. If anything is unclear, nothing is saved.
- **Looking back:** the last reading, the average of each day of the last 7 days, the PDF report in Downloads.
- **Web Dashboard:** a button opens the browser with large charts and the lab table; "Send to the doctor" sends the PDF and a read-only link valid 7 days.

4. Where everything runs

Part	Technology	Where the code is	Where it runs
The app	Kotlin with Jetpack Compose (Android)	<code>android/app/src/main/java/ch/personalhealthy/app/</code>	on the person's phone
The server (API)	TypeScript, one Cloudflare Worker	<code>worker/src/</code> (<code>index.ts</code> is the API)	Cloudflare, close to the person
The database	Cloudflare D1 (SQLite)	tables in <code>worker/schema.sql</code>	Cloudflare, bound to the European Union
The Web Dashboard	plain HTML, CSS and JavaScript	<code>worker/public/my/</code>	in the browser, served by the same Worker
Tests, build, release	GitHub Actions	<code>.github/workflows/</code> , <code>docs/testing/harness/</code>	GitHub
Documents	HTML and Markdown, turned into PDF	<code>docs/</code> (<code>index docs/README.md</code>)	GitHub



The app and the browser never touch the database directly: everything goes through the Worker, which checks who is asking.

What happens when the person saves a reading (the sequence of calls)

1. The app sends `POST /v1/bp/voice` (or `/v1/bp/photo`) to the Worker, signed with the phone's key, with the app version.
2. The Worker checks the signature, that the request is fresh and not a copy, that this app version is allowed, and that the terms were accepted.
3. The Worker writes the three numbers in D1, linked to the person's anonymous account code, and answers "saved".
4. For the Scan, one call comes first: `POST /v1/bp/photo/read`. The Worker checks the trial or the subscription and the daily limit, passes the photo to Anthropic with the owner's key, gets three numbers back and returns them to the phone. Nothing is kept.
5. For the Web Dashboard: the app asks the Worker for a one-time code (`POST /v1/web/code`); the browser opens with it and gets a technical cookie valid 7 days; the page then asks the Worker for the data (`/my/api/data`), which reads them from D1, and draws the charts and the PDF in the browser.

5. Keys and secrets, in plain words

A key is like a house key: whoever has it can open a door. None of them is ever written in the code. They live in GitHub's secret store (encrypted: nobody can read them back, not even the developer, also when the repository is public) or in Cloudflare.

Name	What it opens	Who sets it
CLOUDFLARE_API_TOKEN	lets the release pipeline put the new server online and update the database tables	owner, once
ANDROID_KEYSTORE_B64 , ANDROID_KEYSTORE_PASSWORD , ANDROID_KEY_ALIAS , ANDROID_KEY_PASSWORD	the app's signature: proves every update comes from us, so it installs over the old app. Never replace it: a new one would force everyone to reinstall.	owner, once
KEY_ENCRYPTION_KEY	the server's own secret, created once by the pipeline; it makes the coded fingerprints (Google account, phone, lab files) that cannot be turned back. Never replace it.	pipeline, once
AI_API_KEY (older name ANTHROPIC_API_KEY)	the owner's key for the AI that reads the Scan photos; the spending ceiling is set at the provider	owner
OWNER_CODE	the owner's secret phrase: gives the owner's powers to a new phone of the owner	owner
FAMILY_CODE	the older way for family members to join without Google	owner
PLAY_SERVICE_ACCOUNT	lets the server ask Google Play whether a subscription is paid (once the app is on the Play Store)	owner, later
SECURITY_FIX_TOKEN , CLAUDE_CODE_OAUTH_TOKEN	would let the "Fix" button of the Security console start an automatic fix. Not set , by the owner's choice (it would cost money): fixes are made in the morning routine instead.	not used
The phone's key	signs every request of that phone; created on the phone, stored in the phone's secure chip, never sent anywhere	the app
Cookie <code>hint_s</code>	keeps the browser signed in to the Web Dashboard for 7 days; saved only after consent	the browser

The owner's powers (Admin console, blocking old app versions, settings) work only from the owner's phone on record. If someone stole the owner's Google account and signed in on another phone, they would see that account's data but not the owner's controls.

6. Personal data and compliance

- **Laws:** the EU GDPR and the Swiss data protection law (FADP). Blood pressure and lab results are health data.
- **Consent:** asked explicitly before first use; withdrawn by deleting the account.
- **Pseudonymous, not anonymous:** the data are linked to an account code, never to a name or an email.
- **Where:** Cloudflare D1 in the EU. Anthropic (Scan photo, not kept) and Google (sign-in, payments) are in the US, under their transfer safeguards.
- **How long:** blood-pressure readings at most 365 days (deleted every night); lab results until the person deletes them; error codes 90 days; acceptances of the terms kept as proof.
- **Rights:** see, download, correct and delete one's data, in the app or on the web; answer within a month.
- **Checks:** every control and its evidence is listed in Admin → Observability and in the dossier [docs/compliance/HINT365-Compliance.pdf](https://docs.compliance/HINT365-Compliance.pdf). It is the developer's self-assessment, not legal advice.

7. Automatic jobs (the "crontab")

These jobs run by themselves. Nobody has to start them.

When (Zurich / UTC, summer)	Job	Who runs it	What it does	Goal
every night 05:17 / 03:17	Nightly clean-up	Cloudflare (Worker cron)	deletes readings older than 365 days, expired codes, sessions and doctor links, error codes older than 90 days, trial fingerprints older than 2 years; replaces any old Google id with its fingerprint	keep only what the rules allow
every day 06:10 / 04:10	Security tests	GitHub Actions	vulnerability scan of every library, code scan (Semgrep), secret scan (gitleaks), probe of the live site; opens an issue if something fails	find weaknesses the day they are published
every day 06:30 / 04:30	Error log report	GitHub Actions	reads the errors users met in the last 24 hours; opens an issue if there are any	see problems before users report them
every day 06:48 / 04:48	Morning routine ("daily error-log, security and patching check")	Claude	reads the two issues and the Monday updates, fixes bugs and vulnerabilities it can confirm, merges small library updates, releases, proposes the rest to the owner, reports in the chat	defects, vulnerabilities and patches handled every day
every Monday 05:00 Zurich	Dependabot	GitHub	one pull request for each library of the app and the server with a newer version (the pipeline tools: monthly)	stay up to date
every push to the main code	Build	GitHub Actions	tests, server online, signed app, release, download check	release (section 13)
every pull request	Tests, Android build check, Docs check	GitHub Actions	about 180 tests, the app must compile, documents must match the code	nothing broken reaches people
on the phone, 09:00 and 17:00 local	Reminders	the app	"Remember to measure your blood pressure"; cleared at midnight	help the person keep the diary
on the phone, 2 days before the trial ends	AI trial notice	the app	says the Scan stops unless the person subscribes; voice keeps working	no surprise
by hand	App versions · Owner phone · Promote verified APK	the owner (GitHub → Actions)	block old app versions · give the owner's powers to a new phone · put a previous version back	control in an emergency

8. Logs: what we see and what we do not see

Log	Where	What we see	Kept
Error log	D1 table <code>error_log</code> ; Admin → Observability; the daily issue	an error code, where it happened, the app version, how many times a day	90 days
Event log	D1 table <code>event_log</code>	outcomes as counts: lab report saved or refused and why, Scan retakes and their reason	90 days
Security results	Admin → Security; the <code>security-test</code> issue; the run on GitHub	libraries with known weaknesses, findings in our code, secrets found	last run; reports 30 days
Problem registry	<code>worker/src/ops/problems.json</code> ; Admin → Observability	every real problem met: cause, fix, version, pull request	for ever (in Git)
Pipeline runs	GitHub → Actions	every test, build and release, step by step	as GitHub keeps them
Accounts and money (owner)	Admin console	number of users, Scans and their cost, subscriptions sold, balance	totals per day

What we never see: the values of a reading or a lab result in any log; names, emails or phone numbers; the photos (not kept anywhere); the text people say; the content of the reports. Logs hold only codes and counts linked, at most, to an anonymous account code.

9. Defect management: how a problem gets fixed

1. **Found:** a user meets an error (the app and the web send its code to the error log), or a test fails, or the owner reports it in the chat.
2. **Collected:** every morning the *Error log report* opens an issue with the last 24 hours.
3. **Analysed** at 06:48 by the morning routine: is it a real bug or a normal situation (a wrong phone clock, one blurred photo)?
4. **Recorded** in the problem registry with its cause (status: open).
5. **Fixed** on a branch, with a test that would have caught it; all checks green; merged; released the same day.
6. **Closed:** the registry says fixed, in which version and with which pull request; the issue is closed; the owner gets a short summary.
7. Large or unclear fixes, and anything touching the terms, wait for the owner.

10. Security: vulnerability management

A vulnerability is a known weakness in a piece of software we use. New ones are published every day, so the check runs every day.

When (Zurich / UTC in summer)	What happens	Who
06:10 / 04:10	The <i>Security tests</i> run: every library of the app, the server and the web page is compared with the public list of known weaknesses (OSV.dev); our own code is checked for risky patterns (Sengrep) and for keys written by mistake (gitleaks); the live site is probed. Results go to Admin → Security and, if anything is found, to a GitHub issue labelled <code>security-test</code> .	GitHub, automatic
06:30 / 04:30	The <i>Error log report</i> opens an issue if users met errors in the last 24 hours.	GitHub, automatic
06:48 / 04:48	The morning routine : the developer reads both issues and the scan, decides for each finding whether it really affects the app, and fixes it.	Claude

What the morning routine does with each finding

1. **Takes** the list of open findings.
2. **Checks** where each one is: inside the app that people install, on the server, or only in the tools used to build the app.
3. **Fixes** what is in the app, the server or the web page: updates the library, runs all the tests, releases a new version the same day.
4. **Proposes** to the owner the changes that are too large to do alone (for example a new major version of the Android build tools).
5. **Records** each decision (fixed, waiting for the owner, risk accepted and why) in [docs/security/vulnerability-management.md](#) and removes it from the backlog when the next night's scan confirms it is gone.

Where we are on 02.10.2026. This morning the routine fixed the only findings inside the app (Bouncy Castle, version 0.1.120): from 71 to 59. The 59 left are all in the Android **build tools** (netty, protobuf, commons-io and others brought in by the Android Gradle plugin 8.5.2): they run only on GitHub while the app is built and are not part of the app people install. They disappear with one change: the Android Gradle plugin's major update (issue #94), which waits for the owner's "go" because a major update can change how the app is built. One more (the Kotlin build plugin, issue #97) is an accepted risk until Kotlin 2.x is stable.

The "Fix" button in the Security console is not active: it would need a paid token. Every fix is made by the morning routine at no extra cost.

11. Patch management: keeping libraries up to date

- **Every Monday** GitHub's Dependabot opens a pull request for every library that has a newer version.
- **Small updates** (patch and minor): merged by the morning routine once all checks are green, then released.
- **Large updates** (major): proposed to the owner, because they can change how things work.
- **Security patches** are not kept for Monday: the morning routine applies them the day they appear, with a release the same day.
- After a security release, old versions with the weakness can be blocked so that everyone updates.

12. Change management: how a change is made

1. The owner asks for a change in the chat, in their own words.
2. The developer makes it on a separate branch (`cclaude/...`), never directly on the main code.

3. In the same change the developer updates every document it touches (this document, the guides, the terms, the architecture).
4. A pull request is opened. GitHub runs the checks: about 180 functional and security tests, the app build, the documents check. Nothing can be merged while a check is red.
5. Changes to the terms of use, the privacy policy or the app texts need the owner's approval on GitHub (CODEOWNERS); the owner says "go" in the chat.
6. The change is merged into the main code. That starts the release.

13. Release management: how a new version reaches people

Today (download link)

1. Merge into the main code starts the *Build* pipeline on GitHub.
2. **Tests** again, on the exact code being released.
3. **Server**: the Worker is put online on Cloudflare, the database tables are updated, the secrets are set.
4. **App**: the APK is built and signed with the app's key; the version number goes up by one (0.1.121, 0.1.122...).
5. **Package saved**: a GitHub Release `v0.1.N` keeps the APK and the AAB (the file Google Play wants) for ever.
6. **Check**: the pipeline verifies that the download link serves the new version.
7. The developer sends the link in the chat; the owner installs and tests it; the update installs over the old app, data stay.

When HINT 365 is on Google Play

1. Steps 1–5 above are the same: every release already produces the signed AAB.
2. The AAB is uploaded to the Play Console, first to the **internal testing** track (the owner and a few testers).
3. The owner tests that version from Google Play.
4. The owner promotes it to **production**, with a **staged rollout** (for example 20% of users, then 100% after a day without new errors).
5. Google reviews the version (from a few hours to a few days); then phones update by themselves.
6. The server always accepts the previous versions, so people who have not updated yet keep working.

If something goes wrong

- Stop the rollout in the Play Console, or put the previous verified version back online (*Promote verified APK and server*).
- Block one or more faulty versions on every phone at once (Actions → *App versions*): the app shows only the link to update. Data stay.

14. Costs: what is free, and up to where

As published by the providers in October 2026; the limits can change: check their pricing pages before relying on them.

Service	Free up to	Above that	For HINT 365
Cloudflare Workers (the server)	100,000 requests a day; short computing time per request	Workers Paid, 5 US\$ a month (10 million requests a month included)	a person makes a few dozen requests a day: free well beyond 1,000 people
Cloudflare D1 (the database)	500 MB per database, 5 GB in total; 5 million rows read and 100,000 rows written a day	included in Workers Paid, then by use	today about 0.3 MB; a person adds a few kB a month
GitHub (code, tests, releases)	the repository is public: GitHub Actions minutes are free and unlimited on standard runners; releases are free	—	free
Google sign-in	free	—	free
Google Play	25 US\$ once for the developer account	15% of each subscription	see costs and earnings
Anthropic (Scan)	not free: about 0.2 US cents a photo	stops at the ceiling the owner sets	trial at most 0.09 US\$ a person; subscribers pay 4.99 US\$ a year

Everything except the Scan is free today. The Admin console shows the database size against the free limit, and the money spent on the Scan month by month.

15. Where to find the rest

Document	File
Technical architecture, drawings and flows (6 pages)	docs/architecture/HINT-Architecture.pdf
User guides, Italian and English	docs/HINT-Guida-IT.pdf , docs/HINT-Guide-EN.pdf
Terms of use and privacy policy	docs/legal/HINT365-Terms.pdf , docs/legal/HINT365-Privacy.pdf
Compliance dossier and GDPR record	docs/compliance/HINT365-Compliance.pdf , docs/compliance/gdpr.md
Vulnerability process, in detail	docs/security/vulnerability-management.md , docs/security/vulnerability-flow.pdf
Owner's costs and earnings	docs/business/HINT365-Owner-Economics.pdf
Errors and problems	docs/operations/observability.md

16. Picking the work up again, months later

Everything needed to continue is in the repository on GitHub: the code, the documents, and a diary of every working session. No memory of the chat is needed.

1. **Open an AI coding assistant** that can work on GitHub: Claude Code ([claude.ai/code](#), where this project was built) or another one such as Codex/ChatGPT. Old chats may still be there, but do not rely on them: the repository is the reference.
2. **Give it the repository:** [github.com/comevava76-web/personalhealthy](#).
3. **Paste this request:**

"Read Checkpoint/HANDOVER.md, AGENTS.md, CLAUDE.md and docs/HINT365-How-it-works.pdf. Tell me in a few lines what HINT 365 is, which version is online, what was done last and what was left open.

Then check the error log and the security issues. Do not change anything until I say so."

4. **Read its summary** and decide the next feature or fix, in your own words.
5. The assistant follows the rules already written: change on a branch, documents updated, tests green (section 12), your "go", release (section 13): the download link, Google Play, and in future the Apple App Store (an iPhone version would be a new app, built separately, with the same server).
6. At the end of each session it adds a dated entry to [Checkpoint/HANDOVER.md](#), so the next one starts from there.

To remember	Where
What was done, session by session, and what is open	Checkpoint/HANDOVER.md
The rules of the project (in Italian)	CLAUDE.md , AGENTS.md
How to set up keys and accounts from zero	README.md
All documents, who updates them and when	docs/README.md
Accounts you own	GitHub (code), Cloudflare (server and database), Google Cloud and Google Play (sign-in, store), Anthropic (Scan), the password manager with <code>OWNER_CODE</code>